

Designing scalable frontend boilerplates across Vue, React, and Next.js

A frontend architecture project focused on runtime API safety, UI/logic separation, Storybook-driven UI documentation, and framework-specific state ownership.

ROLE

Frontend Architecture / UI System

TYPE

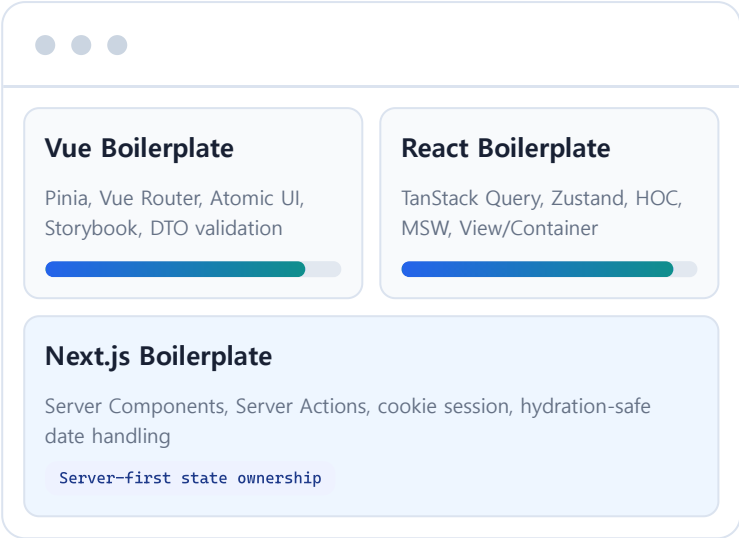
Personal Architecture Project

SCOPE

Vue, React, Next.js Boilerplates

FOCUS

Type Safety, RSC Boundary, State Strategy



SUMMARY

Project Overview

The goal was not to build a single screen, but to reduce repeated setup cost and establish predictable development rules for growing frontend applications.

Challenge 01

Runtime API safety

TypeScript alone cannot validate real API payloads, so DTO validation was added at the API boundary.

Challenge 02

UI and logic separation

React uses View/Container/HOC, Vue uses composable/store, and Next separates server/client ownership.

Challenge 03

Hydration and state ownership

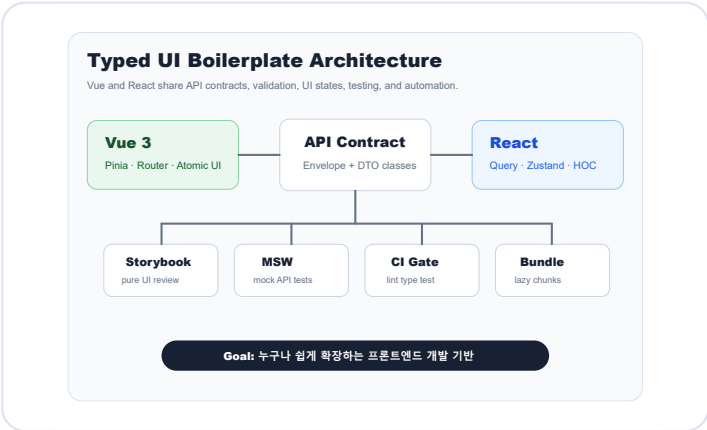
Next.js state is split by source: server data, client cache, UI state, URL state, and mutations.

ARCHITECTURE

Common Design Principles

Each boilerplate follows the same quality standard while respecting its framework runtime. API contracts, state ownership, UI states, testing, and documentation are treated as architecture concerns.

- Validate API envelopes and DTO classes before rendering UI
- Classify failures by frontend contract, backend, network, and auth origin
- Keep loading, empty, and error states reusable
- Prefer lazy route boundaries and scoped imports
- Document pure UI through Storybook and verify contracts through tests

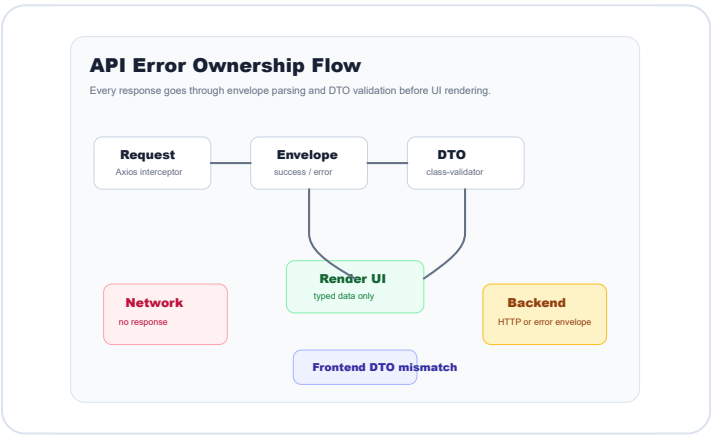


Framework-specific Strategy

Vue 3	
State	Pinia
Routing	Vue Router lazy route
UI	Atomic components + Storybook
Purpose	Fast template-based composition

React	
Server State	TanStack Query
Client State	Zustand
Structure	View, Container, hook, HOC
Purpose	Testable UI/logic separation

Next.js	
Initial Data	Server Component
Client Refresh	TanStack Query
UI State	Zustand
Mutation	Server Action



API RELIABILITY

Typed API and Error Ownership

API responses pass through envelope parsing and DTO validation. A DTO mismatch is treated as a frontend contract issue, while HTTP failures are classified as backend or network issues.

```
{
  "success": true,
  "data": {
    "id": "u-1",
    "email": "demo@example.com"
  }
}
```

RESULT

Verified Outputs

The boilerplates include CI commands, Storybook, DTO tests, client/server boundary documentation, and security audit checks.

CI Gate lint, typecheck, test, production build, Storybook build <code>check:ci</code>	Security moderate-level audit check is expected to pass <code>npm audit</code>	Storybook pure UI components are documented independently <code>Button, Card</code>	MSW mock API contracts support frontend testing <code>handlers.ts</code>	Next Boundary state ownership, RSC rules, hydration-safe date hook <code>BOUNDARY_GUIDE</code>
---	---	--	---	---

TECHNICAL DECISIONS

Key Decisions

Runtime DTO validation

Data is checked at runtime before UI receives it, reducing silent API contract drift.

Next.js state ownership

Initial data stays server-first; TanStack Query is used only for interactive client refresh.

No styled-components

CSS Modules, SCSS Modules, and CSS variables keep Server Components and hydration simpler.

APPENDIX

Project Structure

Folder boundaries are designed to make future AI-assisted development easier to reason about.

```
react-boilerplate
src/components
src/features
src/hoc
src/hooks
src/core
src/test/msw
```

```
boilplate
src/components
src/stores
src/router
src/api
tests/performance
storybook
```

```
next-boilerplate
app
src/features/*/server
src/features/*/actions
src/components/client
src/hoc/server
src/stores
```

NEXT STEP

Expansion Plan

1

Design Token

Connect Figma tokens with CSS variables and framework-specific presets.

2

Generator

Generate feature, DTO, story, test, and route files with consistent conventions.

3

Production auth

Replace demo sessions with production auth providers while preserving server boundaries.